K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING & TECHNOLOGY

2026-27

CAPSTONE ASSIGNMENT

COMPUTER ORGANIZATION & ARCHITECTURE (COA)

COURSE CODE: ENCS302

NAME: Mandeep Singh

ROLL NUMBER: 2301420049

PROGRAMME: BTECH CSE (DS)

SEMESTER: VI

SUBMITTED TO: MS. Mansi Kajal

CASE STUDY ~ 1

PROCESSOR SELECTION FOR SMART TRAFFIC CONTROL SYSTEM

Topic CASE STUDY 1 Date

PROCESSOR SELECTION FOR SMART TRAFFIC CONTROL SYSTEM

Introduction:

Modern cities are increasingly adopting smart traffic control systems as part of intelligence integrated into the already existing transport systems, thus making Intelligent Transport Systems (ITS). These systems utilize sensors, cameras & timers to monitor traffic conditions & dynamically control signal timings. Unlike traditional fixed-time traffic lights, smart systems must respond in real time to changing traffic density

Such a system requires a processor that can:

(i) process multiple inputs simultaneously
(ii) make quick decisions
(iii) operate continuously with moderate power consumption
(iv) ensure reliability & low latency

↳ for example, during peak hours, if vehicle density ↑ in one lane, the system must immediately adjust signal timings to reduce congestion. i.e. An intelligent traffic management system continuously monitors vehicle density using cameras and sensors. As soon as it detects a sharp increase in congestion in that lane, the system instantly recalculates & adjusts the traffic signal timings.

Topic CASE STUDY 1 Date

Problem Understanding

In modern intelligent traffic management system, the core challenge is to dynamically control traffic signals at an intersection based on actual traffic conditions rather than using fixed timers. During peak hours, vehicles density can change suddenly - one lane may become heavily congested while others remain relatively free. If the signal timings are not adjusted immediately, it leads to long queues, increasing waiting time, fuel wastage, higher ~~pollutions~~ pollution and even risk of accidents.

- Functional Requirements:

(i) Continuous collection of sensor data
(ii) Real time decision making
(iii) Handling multiple input streams

- Non functional Requirements:

(i) Low Latency
(ii) Energy efficiency
(iii) High Reliability & fault tolerance

(P.T.O.)

Topic CASE STUDY - 1 Date

COA Concepts applied:

(A) Functional Units of a computer:

- Input Unit: receives data from sensors & cameras
- Control Unit: controls execution of instructions & manages signal switching
- ALU: Performs logical operations for traffic decision-making
- Registers: store temporary data for quick access
- Memory Unit: stores traffic patterns & system states
- Output Unit: sends signals to traffic lights

(B) RISC vs CISC Architecture:

Feature	RISC	CISC
Instruction Type	Simple	Complex
Execution Speed	fast	slower
Pipeline Efficiency	↑	↓
Power Consumption	↓	↑

∴ RISC architecture is ~~preferred~~ preferred

(C) Instruction Throughput (IPC):

Instruction throughput refers to the number of instructions executed per cycle, higher IPC → faster decision making, very important for handling multiple sensor inputs simultaneously

(D) Instruction-Level Parallelism:

↳ Allows multiple instructions to be executed at the same time improving system efficiency & reducing delays.

Topic CASE STUDY - 1 Date

Design Analysis & Justification

- Processor choice: Pipelined RISC Processor

Pipelining divides instruction execution into stages:

(i) Instruction Fetch (IF)
(ii) Instruction Decode (ID)
(iii) Execute (EX)
(iv) Memory Access (MEM)
(v) Write Back (WB)

IF → ID → EX → MEM → WB

- Benefits of Pipelining:

(i) Multiple instructions processed simultaneously
(ii) Increased throughput
(iii) Better CPU utilization

Pipelining Hazards & Solutions

Hazard	Description	Solution
· Data Hazard	Instruction dependency	Data Forwarding
· Control Hazard	Branching Decisions	Branch prediction
· Structural Hazard	Resource conflict	Hardware duplication

(PTO →)

(PTO →)
(Back to this page)

CASE STUDY 1

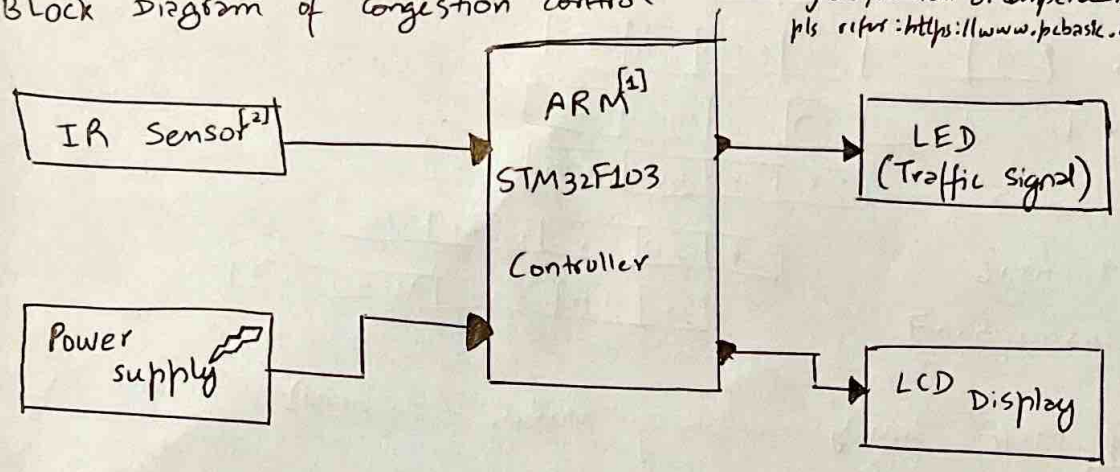
Diagrams:

figure glossary:

↳ fig (1) :- [1] The STM32F103 controller is a 32-bit microcontroller known for low power consumption, high performance & extensive peripherals set, for more info pls refer: https://reversepcb.com/stm32f103/

→ [2] IR Sensor is a device that detects infrared radiation to sense objects, motion or temperature changes. for more info pls refer: https://www.pcbasic.com/blog/ir_sensors.html

(I) Block Diagram of congestion control



(II) Block Diagram of Ambulance clearance

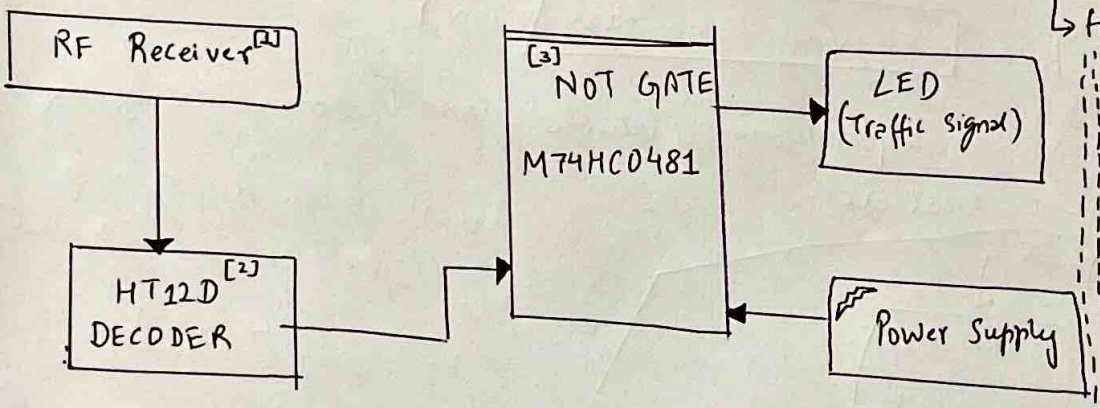


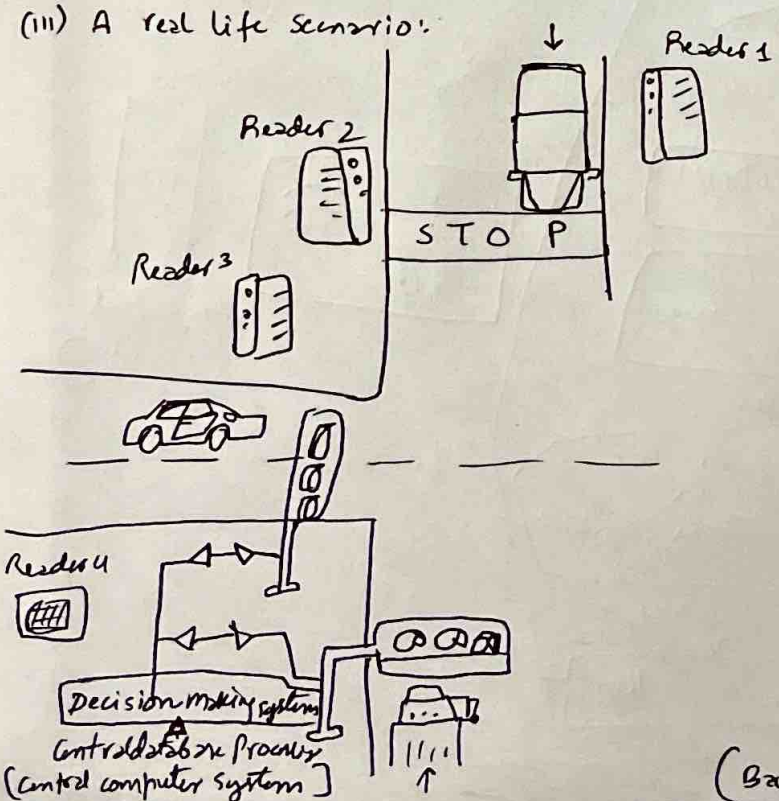
figure glossary:-

↳ fig(II): [1] RF Receiver is a device that receives & processes radio frequency (RF) signals to extract the original info. For more info pls refer: https://www.numberanalytics.com/blog/ultimate-guide-rf-receiver-rf-engineering

[2] HT12D Decoder is a CMOS low scale integrated circuit (CMOS = complementary Metal oxide semiconductor), designed for remote control system applications, capable of decoding 12bits of info, with 8 address bits & 4 data bits. for more info pls refer: https://www.senstry.com/blog/holtek-ht12d-encoding-and-decoding-ic-working-principle-circuit-connection-pdf-data-sheet-and-pricing.html.

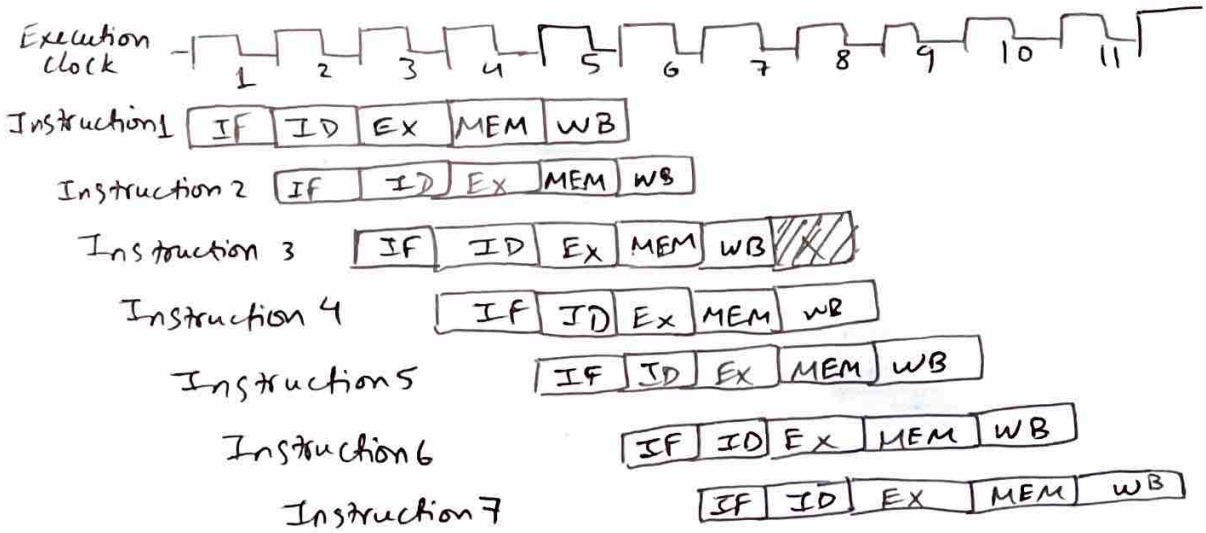
[3] M74HC0481 is a high speed CMOS hex inverter, functioning as a NOT GATE that inverts logic signals with low power consumption & high noise immunity, each inverter outputting four complement of its input. for more info pls refer: https://users.ece.utexas.edu/~valvano/Datasheets/74HC04.pdf

(III) A real life scenario:

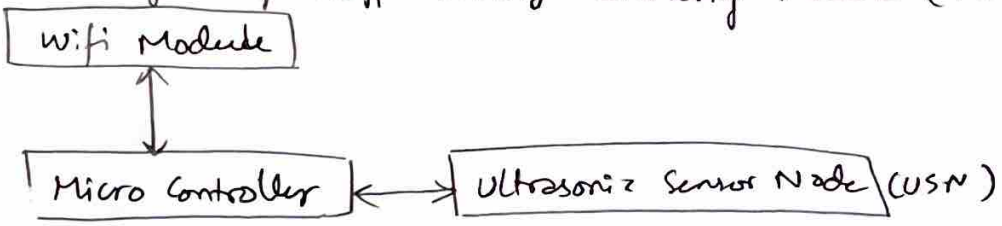


(P10→)
(Back of this page)

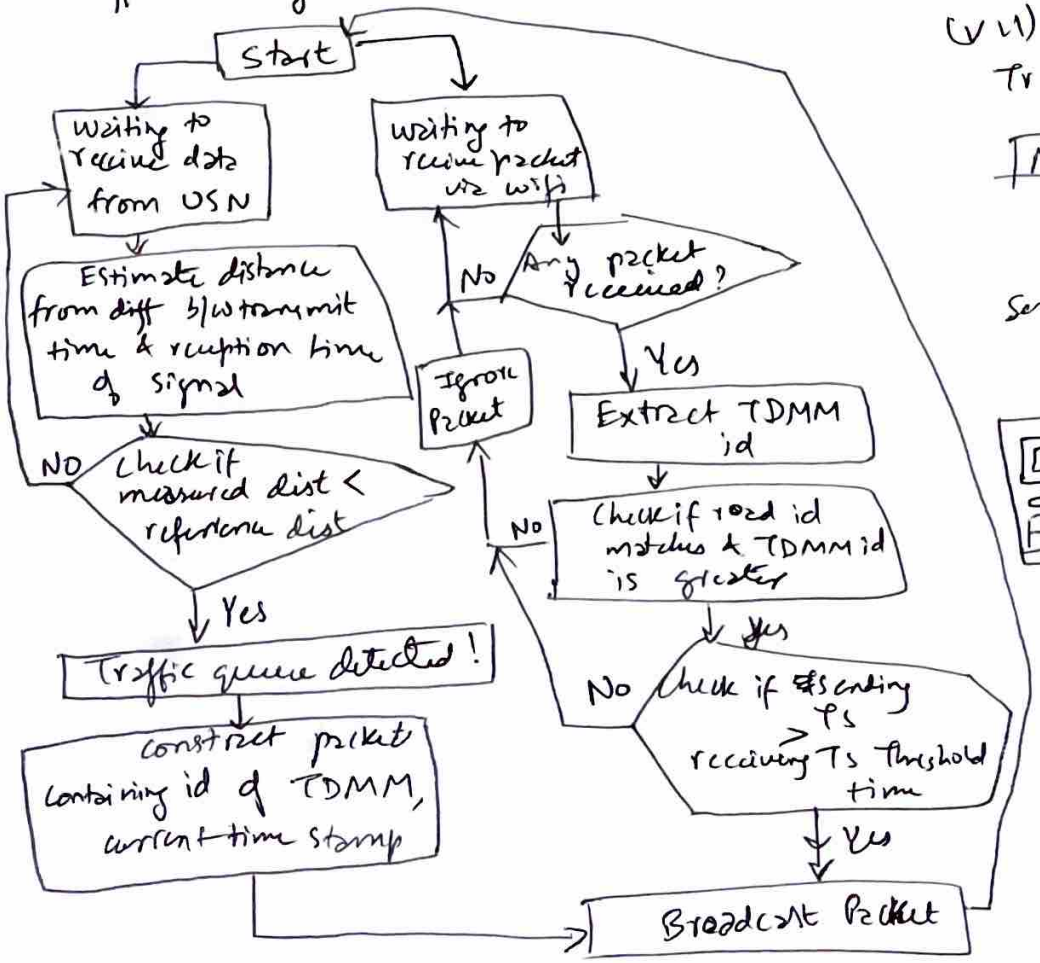
(iv) Instruction execution in 5-stage pipeline:



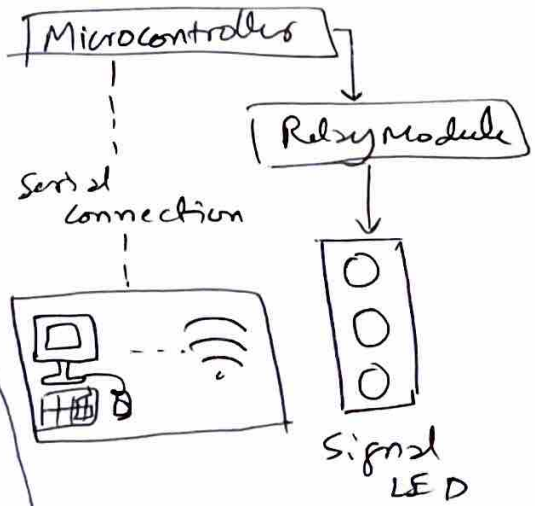
(v) Block Diagram of Traffic density monitoring module (TDMM)



(vi) Work flow diagram of detecting presence of vehicle queue & communicating it to Traffic management module:



(vii) Block diagram of Traffic management module



Topic CASE STUDY - 1 Date

Final Recommendation:

- A pipelined RISC processor is used because:

(i) ensures fast real-time processing
(ii) improves instruction throughput
(iii) Reduces power consumption
(iv) Handles multiple inputs efficiently

Final Conclusion:

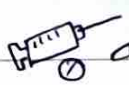
- The smart traffic control system demands a processor that delivers high speed, efficiency & reliability to handle real-time data from sensors & cameras while making instant decisions.
- A pipelined RISC architecture stands out as the most suitable choice due to its ability to execute multiple instructions simultaneously with minimal latency. This architecture ensures fast processing of the traffic data, quick decision-making for signal control & efficient resource ~~utilization~~ utilization.
- By combining simplicity, high throughput & low power consumption, a pipelined RISC-based processor provides ideal balance of performance & reliability required for modern intelligent traffic management systems.

CASE STUDY ~ 2

I/O DESIGN FOR HOSPITAL PATIENT MONITORING UNIT

Topic CASE STUDY -2 Date

I/O DESIGN FOR HOSPITAL PATIENT MONITORING UNIT



Introduction:

A hospital patient monitoring system continuously tracks vital parameters such as heart rate, oxygen level, & body temperature.

Since, nowadays people suffer from atleast one disease at one point in their lives and in hospital, difficulty occurs in taking care of the patients.

The underlying motivation for this design stems from the aspiration to address the challenges faced by elderly & physically impaired individuals in monitoring patients both at home & in hospitals.

Additionally, it acknowledges the difficulties that disabled or elderly individuals may encounter in remembering the time & dosage of their medications.

Since non-compliance with medication instructions is prevalent among significant number of patients, particularly the elderly & the disabled. This non-compliance leads to various issues such as missed doses, taking wrong medication, incorrect dosage leading to ineffective treatment

Even the caregivers such as nurses, occasionally require reminders to ensure patients take their meds at ~~predetermined~~ predetermined times.

Topic CASE STUDY - 2 Date

Problem Understanding

A hospital patient monitoring system is a safety-critical real-time system designed to continuously track vital physiological parameters such as heart rate, oxygen saturation (SpO_2) & body temperature. These parameters are collected through medical sensors & must be processed instantly to detect abnormal conditions.

The key challenge in this system is the continuous & high-frequency data flow from multiple sensors. The processor must handle this data without delays, as even a slight lag in detecting critical changes can have serious consequences for patient safety.

Unlike typical computing systems, this environment imposes strict requirements:

- Real-time responsiveness (immediate alert generation)
- High Reliability & accuracy (no data loss or incorrect readings)
- Efficient resource utilization (no CPU overload)

Another major constraint is that the system must support simultaneous monitoring of multiple patients, increasing volume of incoming data & requiring efficient data transfer ~~mechanisms~~ mechanisms, the system must also ensure:

- Low latency in data transfer from sensors to memory
- Minimal CPU involvement in routine data handling
- Continuous operation without bottlenecks

∴ Core problem is designing an I/O system that can handle continuous real-time data ~~streams~~ streams, reduced CPU overload ensure fast & reliable data transfer, support timely detection & alert generation.

Topic CASE STUDY - 2 Date

COA Concepts Applied

(A) Memory - Mapped I/O vs I/O - Mapped I/O

* Memory - Mapped I/O
  ↳ Idea

I/O devices are treated like normal memory locations.
i.e. devices are assigned addresses in main memory, CPU uses normal instructions (LOAD / STORE) to access them.

* I/O - Mapped I/O (Isolated I/O)
  ↳ Idea

I/O devices are kept in a separate address space
i.e. Memory & I/O have different address ranges, CPU uses special instructions (like IN, OUT)

→ Advantages	Memory - Mapped I/O	Isolated I/O
	(i) Faster Access (ii) No special instructions needed (iii) Easier Programming (iv) Full instruction set can be used	(i) Memory space is preserved (ii) Clear separation of memory & devices
→ Disadvantages	(i) uses part of memory space (ii) slightly reduces available RAM	(i) Slower (ii) limited instructions (iii) Less flexible

Topic CASE STUDY -2 Date

* Comparison (Memory-Mapped I/O vs Isolated I/O)

Feature	Memory-Mapped I/O	Isolated I/O
• Address space	shared with memory	separate
• Instructions used	Normal (LOAD/STORE)	Special (IN/OUT)
• Speed	Faster	slower
• Flexibility	High	Limited
• Memory Usage	Uses memory space	No memory usage

Note: Since Memory-Mapped I/O is more flexible & efficient, it is considered as a suitable candidate for real-time systems whereas the Isolated I/O provides separation at the cost of speed & flexibility

⟶ ∴ Memory-Mapped I/O is chosen.

(B) I/O Techniques

Input/output techniques define how CPU interacts with external devices such as sensors in our patient monitoring system. The three primary techniques are programmed I/O, interrupt-driven I/O & Direct Memory Access (DMA) each differing in terms of CPU involvement & efficiency.

→ In Programmed I/O, CPU is directly responsible for checking whether a device is ready to send or receive data. This is done through continuous polling, where CPU repeatedly checks device status in a loop. As a result CPU remains busy waiting & cannot perform other useful tasks leading to inefficient utilization of processor resources & increased latency

∴ PROGRAMMED I/O is UNSUITABLE.

Topic CASE STUDY - 2 Date

→ Interrupt-driven I/O, improves efficiency by allowing CPU to perform other ~~tst~~ tasks while waiting for the device. When device is ready, it sends an interrupt signal to CPU, which temporarily pauses its current execution to handle data transfer, thus reducing unnecessary CPU waiting & improves overall performance however in systems with continuous & high-frequency data input, such as patient monitoring system, frequent interrupts overload CPU & reduce efficiency ∴ INTERRUPT-DRIVEN I/O IS STILL UNSUITABLE.

→ Direct Memory Access (DMA) is the most efficient method where a dedicated DMA controller handles the transfer of data directly b/w the I/O device & main memory without continuous ~~CPU~~ CPU involvement, the CPU initiates transfer & is notified when once it is complete, thus greatly reducing CPU overload ∴ ensuring faster data transfer, low latency & efficient system performance ∴ DMA IS SUITABLE!

Design Analysis & Justification

DMA allows direct data transfer between devices & memory without continuous CPU involvement

- Working: (i) Sensors collect patient data
  (ii) DMA ~~&~~ controller transfers data directly to memory
  (iii) CPU processes data only when required
  (iv) Alerts generated if abnormal values detected

Sensor → DMA → Memory → CPU → Alert

- Additional Concept: Buffering

Buffering temporarily stores incoming data before processing: (i) Prevents data loss, (ii) ensures smooth data flow

#DIAGRAMS:

(1) BLOCK DIAGRAM: (OVER ALL)

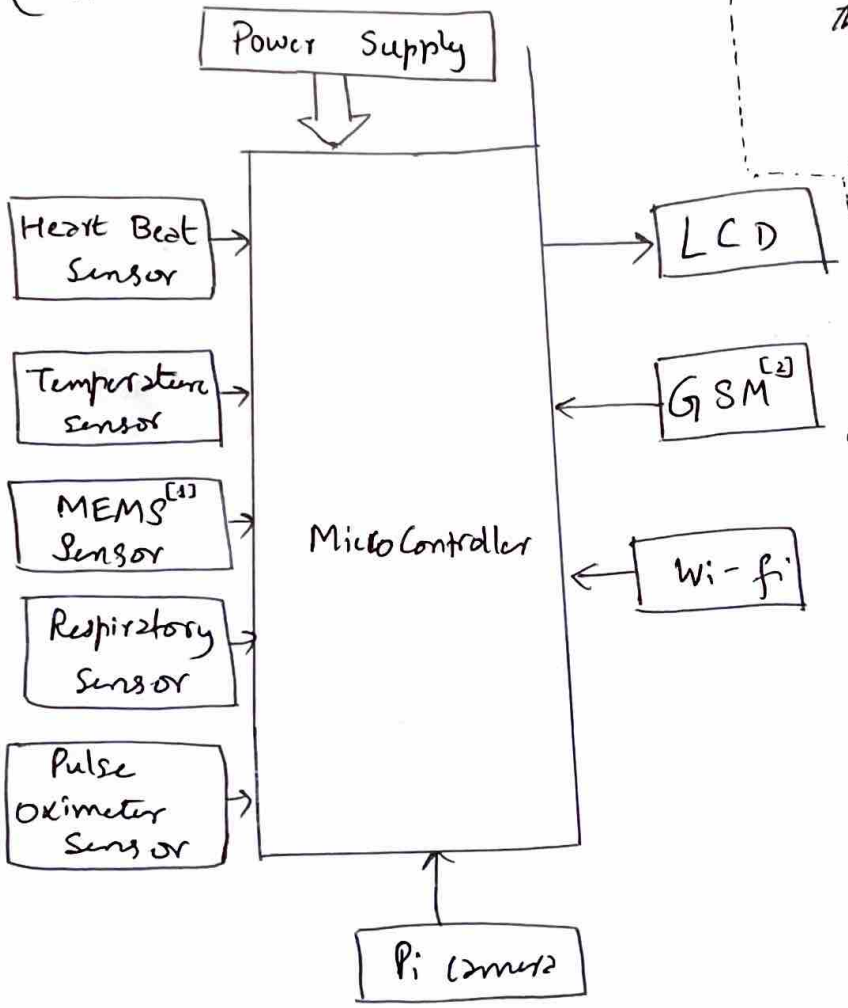
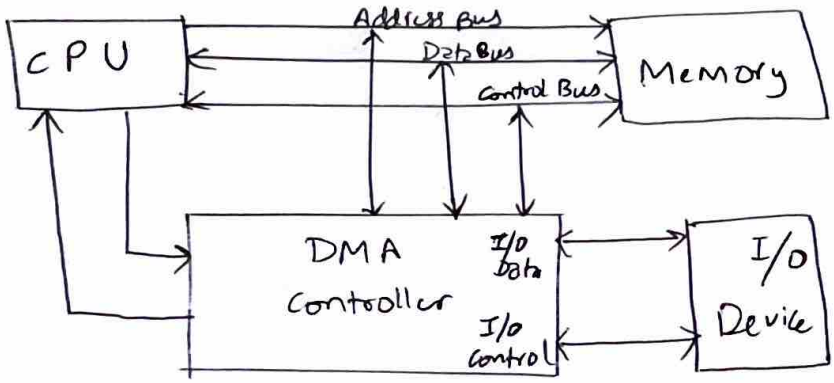


figure glossary:

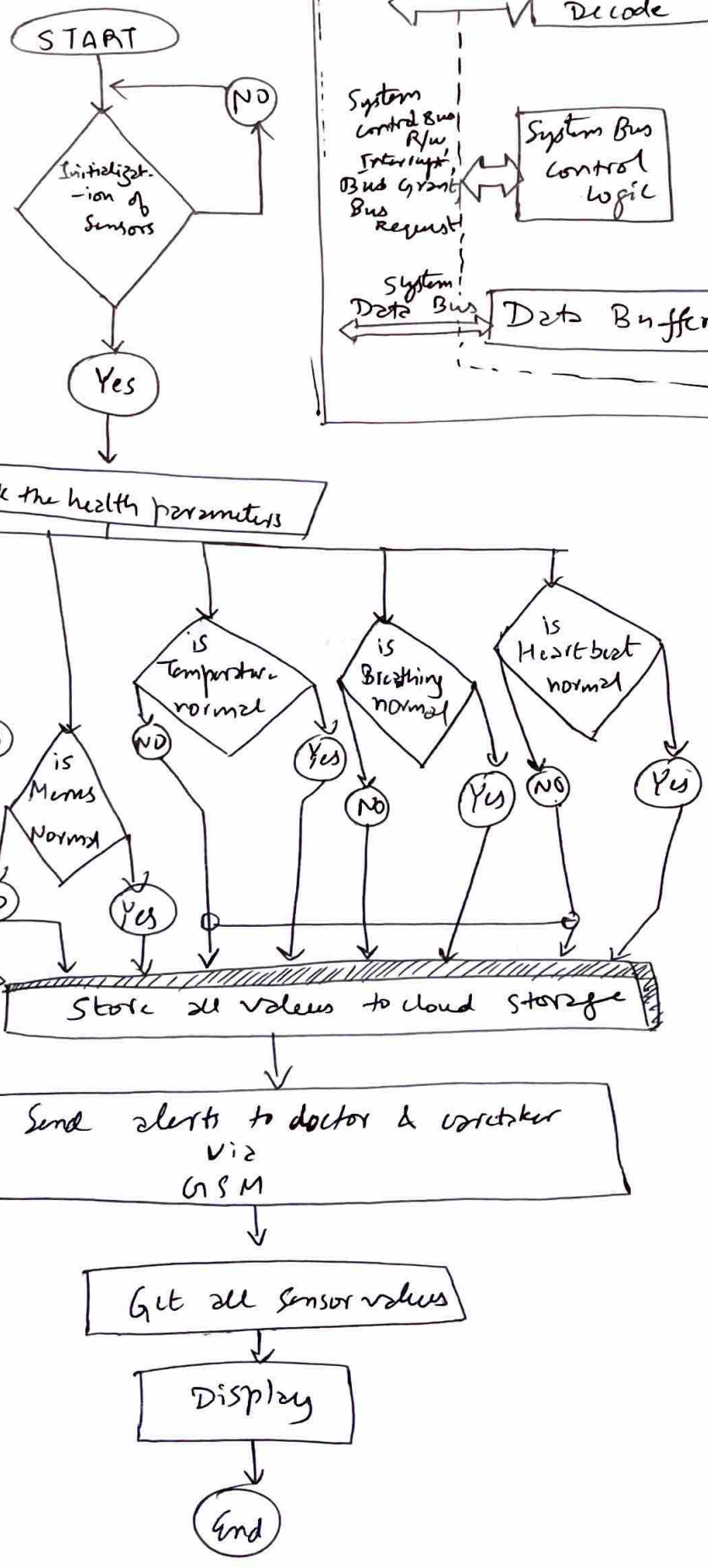
fig(1): [1] MEMS Sensor ~ called Micro Electro Mechanical sensors in short. It is an integration of both active & passive components into single silicon substrate with help of advanced IC manufacturing technologies. The active components being sensors & Actuators while passive components being electronic systems & micro systems. for more info pls refer: " https://www.electronicshub.org/mems-sensors/ "

[2] GSM: Global System for Mobile communications, a digital standard for mobile networks that enable voice, text & data transmission using time & frequency division techniques. for more info pls refer: " https://www.rfwireless-world.com/tutorials/gsm-tutorial-basics-architecture-interfaces-protocol-stack "

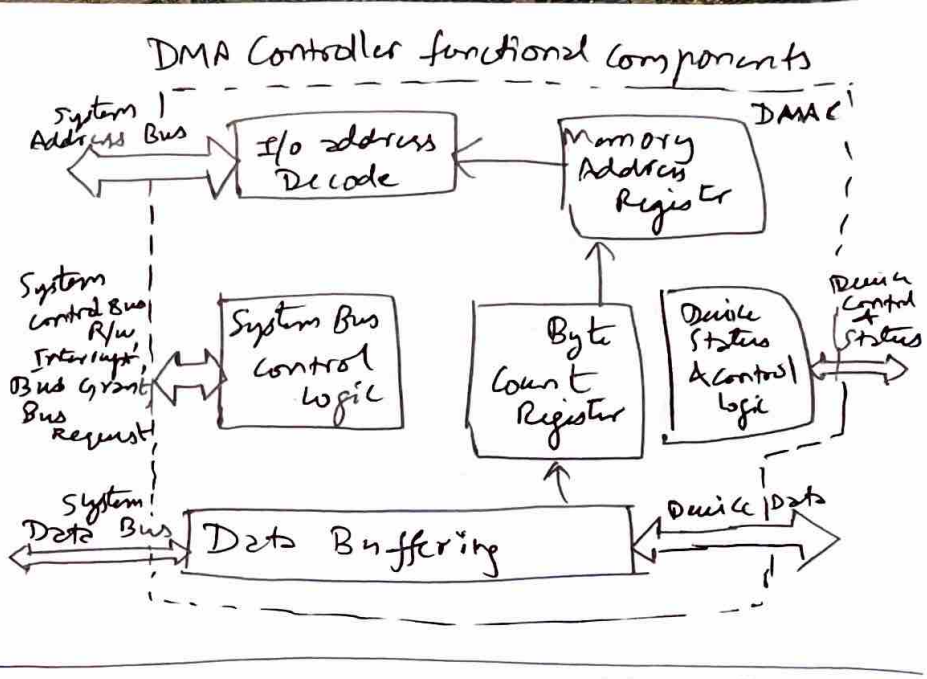
(11) BLOCK DIAGRAM DMA CONTROLLER



FLOW CHART



DMA Controller functional components



Topic CASE STUDY - 2 Date

Final Recommendation:

Use:

- Memory-Mapped I/O
- DMA-based data transfer

This ensures:

- Efficient CPU usage
- Real-time response
- Reliable monitoring

Conclusion:

In healthcare systems, timely & accurate data processing is essential for patient safety. The use of DMA reduces CPU involvement & enables efficient handling of continuous data streams, while memory-mapped I/O ensures faster & more flexible communication with devices. Together, they provide a reliable, low latency, & efficient solution for real-time patient monitoring.